IT UNIVERSITY OF COPENHAGEN

Functional Programming

Jesper Bengtson

Modules and .fsi-files

Sir Charles Antony Richard Hoare

- Brittish computer scientist with foundational work in programming languages, operating systems, program verification, …
- Turing Award recipient
- Inventor of quick sort
- Inventor of the null pointer
- ALGOL / CSP / Hoare Logic
- Dining philosophers problem

Tony Hoare

1934 - 2026

The null pointer

I call it my billion-dollar mistake. It was the invention of the null reference in 1965. At that time, I was designing the first comprehensive type system for references in an object oriented language (ALGOL W). My goal was to ensure that all use of references should be absolutely safe, with checking performed automatically by the compiler. But I couldn't resist the temptation to put in a null reference, simply because it was so easy to implement. This has led to innumerable errors, vulnerabilities, and system crashes, which have probably caused a billion dollars of pain and damage in the last forty years

Last two weeks

We covered data types and collections

- Discriminated unions
- Maps
- Sets
- Data representation
- The Option type
- The Result type
- Mutable data

Last two weeks

We covered data types and collections

- Discriminated unions
- Maps
- Sets
- Data representation
- The Option type
- The Result type
- Mutable data

Arithmetic expressions

```
let rec evalA arith =
  match arith with
   | N n -> fun _ -> n
   | V v -> Map.find v
   | Add(a, b) ->
       fun st ->
           (+)
           (evalA a st)
           (evalA b st)
  …
```

```
let binop : (int -> int -> int)
                -> (state -> int)
                -> (state -> int)
                -> state -> int =
    fun f x y s -> f (x s) (y s)

let rec evalA arith =
  match arith with
   | N n          -> fun _ -> n
   | V v          -> Map.find v
   | Add(a, b) -> binop (+)
                        (evalA a)
                        (evalA b)
```

Arithmetic expressions

```
let rec evalA arith =
  match arith with
   | N n -> fun _ -> n
   | V v -> Map.find v
   | Add(a, b) ->
       fun st ->
           (+)
           (evalA a st)
           (evalA b st)
  …
```

```
let binop : (int -> int -> int)
                -> (state -> int)
                -> (state -> int)
                -> state -> int =
    fun f x y s -> f (x s) (y s)

let rec evalA arith =
  match arith with
   | N n          -> fun _ -> n
   | V v          -> Map.find v
   | Add(a, b) -> binop (+)
                        (evalA a)
                        (evalA b)
```

Arithmetic expressions

```
let binop : (int-> int -> int)
                -> (state -> int)
                -> (state -> int)
                -> state -> int =
    fun f x y s -> f (x s) (y s)

binop (+) (evalA a) (evalA b)
```

Arithmetic expressions

```
let binop : (int-> int -> int)
                -> (state -> int)
                -> (state -> int)
                -> state -> int =
    fun f x y s -> f (x s) (y s)

binop (+) (evalA a) (evalA b) =
(fun f x y s -> f (x s) (y s)) (+) (evalA a) (evalA b)
```

Arithmetic expressions

```
let binop : (int-> int -> int)
                -> (state -> int)
                -> (state -> int)
                -> state -> int =
    fun f x y s -> f (x s) (y s)
```

```
binop (+) (evalA a) (evalA b) =
(fun f x y s -> f (x s) (y s)) (+) (evalA a) (evalA b) ↝
(fun x y s -> (+) (x s) (y s)) (evalA a) (evalA b)
```

Arithmetic expressions

```
let binop : (int-> int -> int)
              -> (state -> int)
              -> (state -> int)
              -> state -> int =
    fun f x y s -> f (x s) (y s)
```

```
binop (+) (evalA a) (evalA b) =
(fun f x y s -> f (x s) (y s)) (+) (evalA a) (evalA b) ↪
(fun x y s -> (+) (x s) (y s)) (evalA a) (evalA b) ↪
(fun y s -> (+) (evalA a s) (y s)) (evalA b)
```

Arithmetic expressions

```
let binop : (int-> int -> int)
                -> (state -> int)
                -> (state -> int)
                -> state -> int =
    fun f x y s -> f (x s) (y s)
```

```
binop (+) (evalA a) (evalA b) =
(fun f x y s -> f (x s) (y s)) (+) (evalA a) (evalA b) ↪
(fun x y s -> (+) (x s) (y s)) (evalA a) (evalA b) ↪
(fun y s -> (+) (evalA a s) (y s)) (evalA b) ↪
fun s -> (+) (evalA a s) (evalA b s)
```

Arithmetic expressions

```
let binop : (int-> int -> int)
          -> (state -> int)
          -> (state -> int)
          -> state -> int =
    fun f x y s -> f (x s) (y s)
```

```
binop (+) (evalA a) (evalA b) =
(fun f x y s -> f (x s) (y s)) (+) (evalA a) (evalA b) ⇝
(fun x y s -> (+) (x s) (y s)) (evalA a) (evalA b) ⇝
(fun y s -> (+) (evalA a s) (y s)) (evalA b) ⇝
fun s -> (+) (evalA a s) (evalA b s) ⇝*
fun s -> eval a s + evalA b s
```

Arithmetic expressions

```
let binop : (int-> int -> int)
                -> (state -> int)
                -> (state -> int)
                -> state -> int =
    fun f x y s -> f (x s) (y s)
```

Note that the actual type of binop is

```
('a -> 'b -> 'c) -> ('d -> 'a) -> ('d -> 'b) -> 'd -> 'c
```

This week

- Record types (recapitulation)
- Modules
- More maps (the assignment)
- Some more programming examples

This weeks assignment

Create support for memory management

- Allocate memory
- Deallocate (free) memory
- Set a memory cell to a certain value
- Get the value from a memory cell

This weeks assignment

Think of memory as an array where the indexes are memory addresses and the individual cells store information in memory

- Allocation allows addresses of the memory to be used (map address to default value 0)
- Deallocation prevents addresses of the memory to be used (remove address from map)
- Reading from and writing to a memory address is possible only if that address has been allocated

The green exercises works with maps

Questions?

Records

Records allow us to structure data

```
type person =
   { firstname : string;
     lastname  : string;
     age       : int }
```

Records can be of arbitrary size

Records

Records are declared by providing input to all of their fields
(no partial application)

```
let Jesper =
    { firstname = "Jesper";
      lastname = "Bengtson";
      age = 41}
val Jesper : person =
 { firstname = "Jesper";
    lastname = "Bengtson";
    age = 41;}
```

Records

Records are declared by providing input to all of their fields
(no partial application)

All field names must match with an already declared type

```
let Jesper =
    { firstname = "Jesper";
      lastname = "Bengtson";
      age = 41}
val Jesper : person =
 { firstname = "Jesper";
    lastname = "Bengtson";
    age = 41;}
```

Records

Individual fields are accessed using their name

```
jesper.firstname
val it : string = "Jesper"

jesper.lastname
val it : string = "Bengtson"

jesper.age
val it : int = 41
```

let-patterns

It is possible to obtain values from compound statements

```
let { firstname = fn;
      lastname = ln;
      age = x} = jesper in
    (fn, ln, x)
val it : string * string * int =
    (“Jesper", "Bengtson", 41)
```

Creating records

Records can be created by providing all arguments

```
> let Jesper =
    { firstname = "Jesper"; lastname = "Bengtson"; age = 41}
- val Jesper : person =
    { firstname = "Jesper"; lastname = "Bengtson"; age = 41}
```

Creating records

Records can be created by providing all arguments

```
> let Jesper =
    { firstname = "Jesper"; lastname = "Bengtson"; age = 41}
- val Jesper : person =
    { firstname = "Jesper"; lastname = "Bengtson"; age = 41}
```

Or by updating existing ones

```
> let OldJesper =
    { Jesper with age = 48}
- val OldJesper : person =
    { firstname = "Jesper"; lastname = "Bengtson"; age = 47}
```

Use records for state

I highly recommend you use records in State.fs
to store your program state

Use records for state

I highly recommend you use records in State.fs
to store your program state

Today you just have the variable store.
This week you add memory
In the coming weeks.... who knows?

Use records for state

I highly recommend you use records in State.fs
to store your program state

Today you just have the variable store.
This week you add memory
In the coming weeks.... who knows?

Records are relatively easy to extend

Modules

- Modular programming design
  - Encapsulation
  - Abstraction
  - Reuse of components
- A module is characterised by
  - A signature (.fsi -file)
  - A matching implementation (.fs file)

Signatures

Signatures are given in .fsi-files

```
module ModuleName
 type T  (required type)
 val f : <type> (required function)
 val g : <type> (required function)
 …
 …
```

.fsi-files list the types (if any) that must be defined, and the visible functions (if any) that must be implemented

Implementations

Implementations are given in .fs-files

```
module ModuleName (same as .fsi)
 type T = <def> (must be discrete
                 union or record)
 let f = <implementation>
 let g = <implementation>
 …
 …
```

implementation types must match .fsi types

Implementations

Implementations are given in .fs-files

```
module ModuleName (same as .fsi)
 type T = <def> (must be discrete
                 union or record)
 let f = <implementation>
 let
 …
 …
```

Important! You will get very weird error messages if you say ‘type T = int’, for instance

Putting it together

Signatures are given in .fsi-files

```
module ModuleName
 type T
 val f : <type>
 val g : <type>
 …
 …
```

Implementations are given in .fs-files

```
module ModuleName
 type T = <def>
 let f = <implementation>
 let g = <implementation>
 …
 …
```

Putting it together

Signatures are given in .fsi-files

```
module ModuleName
 type T
 val f : <type>
 val g : <type>
 …
 …
```

Implementations are given in .fs-files

```
module ModuleName
 type T = <def>
 let f = <implementation>
 let g = <implementation>
 …
 …
```

Note that .fsi uses val and .fs uses let

Rational numbers

Let’s write a small library for rational numbers

- Create a new project
- Create an .fsi file and define the signatures
- Create an .fs file and write the implementation

Title Text

Recall that

$$\frac{a}{b} + \frac{c}{d} = \frac{ad + bc}{bd} \qquad \frac{a}{b} - \frac{c}{d} = \frac{ad - bc}{bd}$$

$$\frac{a}{b} * \frac{c}{d} = \frac{ac}{bd} \qquad \frac{a}{b} / \frac{c}{d} = \frac{ad}{bc}$$

Let’s code

Rational1.fsi

```
module Rational
    type rat

    val mkRat : int -> int -> rat

    val ( .+. ) : rat -> rat -> rat
    val ( .-. ) : rat -> rat -> rat
    val ( .*. ) : rat -> rat -> rat
    val ( ./. ) : rat -> rat -> rat
```

Rational1.fs

```
module Rational
  type rat = R of int * int

  let mkRat a b = R (a, b)

  let ( .+. ) (R (a, b)) (R (c, d)) =
      R (a * d + b * c, b * d)
  let ( .-. ) (R (a, b)) (R (c, d)) =
      R (a * d - b * c, b * d)
  let ( .*. ) (R (a, b)) (R (c, d)) =
      R (a * c, b * d)
  let ( ./. ) (R (a, b)) (R (c, d)) =
      R (a * d, b * c)
```

Problem

We do not have unique representations of rational numbers

$$\frac{1}{2} = \frac{2}{4} = \frac{4}{8} = \frac{8}{16} = \ldots$$

Solution

Euclid's algorithm

$$\gcd\ a\ 0 = a$$
$$\gcd\ a\ b = \gcd\ b\ (a \bmod b)$$

One of the oldest algorithms we know of (c. 300BC)

Let’s code

Rational.fsi

```
module Rational
    type rat

    val mkRat : int -> int -> rat

    val ( .+. ) : rat -> rat -> rat
    val ( .-. ) : rat -> rat -> rat
    val ( .*. ) : rat -> rat -> rat
    val ( ./. ) : rat -> rat -> rat
```

Rational.fsi

```
module Rational
    type rat

    val mkRat : int -> int -> rat

    val ( .+. ) : rat -> rat -> rat
    val ( .-. ) : rat -> rat -> rat
    val ( .*. ) : rat -> rat -> rat
    val ( ./. ) : rat -> rat -> rat
```

Exactly the same as before

Rational.fs

```
module Rational
  type rat = R of int * int

  let rec gcd a =
     function
     | 0 -> a
     | b -> gcd b (a % b)

  let mkRat a b =
     let g = gcd a b
     R (a / g, b / g)

  let ( .+. ) (R (a, b)) (R (c, d)) =
      mkRat (a * d + b * c) (b * d)
...
```

Program.fs

```
module Rational

printfn "%A" (mkRat 12 2 .+. mkRat 12 4)
printfn "%A" (mkRat 12 2 .-. mkRat 12 4)
printfn "%A" (mkRat 12 2 .*. mkRat 12 4)
printfn "%A" (mkRat 12 2 ./. mkRat 12 4)
```

Final problem

When we print we expose implementation details
(`R (a, b)` should be internal to the module)

Final problem

When we print we expose implementation details
(`R (a, b)` should be internal to the module)

This is because of the ToString method that defaults to pretty-printing discriminated unions in a nice way

Solution

Override the ToString method

Let’s code

Rational.fs

```
module Rational
  type rat =
     R of int * int
        override q.ToString() =
            match q with
            | R (a, b) -> sprintf "(%d / %d)" a b
...
```

```
printfn "%A" (mkRat 3 6)
```

Now outputs “(1 / 2)”

Questions?